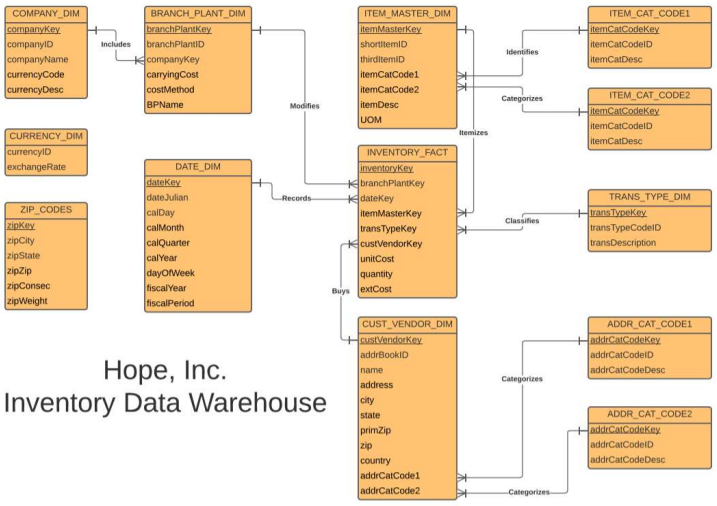
2/15/24

LAB #2 Subtotal Operators Assignment

- *DO NOT USE INNER JOIN, FULL JOIN, LEFT JOIN, RIGHT JOIN IN YOUR SOLUTIONS.*
- *use WHERE clause in connecting the primary and foreign keys of the tables.*



Query 1: Sales Order Shipments by Month and Category Code1

Display the sum of the extended cost and the sum of the quantity.
The results should include data for shipments (transaction type 5) in calendar year 2011. Summarize the result by calendar month and Address Category Code 1. The result should include the grouped columns and the full totals for every combination of grouped columns. Do not use the GROUPING SETS and UNION operators.

```
--Query 1: Sales Order Shipments by Month and Category Code1



SELECT
   calMonth,
   addrCatCode1,
   SUM(extCost) AS Total_Cost,
   SUM(quantity) as Total_Quantity
FROM
   date_dim dd,
   inventory_fact inf,
```

```
   addr_cat_code1 cd,
   CUST_VENDOR_DIM cd
WHERE
   dd.dateKey = inf.dateKey
   AND cd.addrCatCode1 =cd.addrCatCodeKey
   AND inf.CustVendorKey = cd.CustVendorKey
   AND dd.CalYear = '2011'
   AND inf.TransTypeKey = 5
GROUP BY
   CUBE(calMonth,addrCatCode1)
ORDER BY
   calMonth,addrCatCode1;
```

----------RESULTS:----------

History Results

Result set 1 Details

Filter table Total:78

CALMONTH	ADDRCATCODE1	TOTAL_COST	TOTAL_QUANTITY
1	1	12876795.00	12970.0000
1	2	6856150.00	7004.0000
1	3	5643236.00	5629.0000
1	4	5657775.00	5691.0000
1	5	8189894.00	8368.0000
1	-	39223850.00	39662.0000
2	1	9481319.00	8983.0000
2	2	6808224.00	6935.0000
2	3	4023869.00	4139.0000
2	4	4862270.00	4657.0000
2	5	5838930.00	6147.0000

Items per page: 50 1–50 of 78 items 1 of 2 pages

Query 2: Sales Order Shipments by Name, Zip, and Quarter

Write an SQL statement to display the sum of the extended cost and the number of inventory transactions. The results should include data for shipments (transaction type 5) in calendar years 2011 and 2012. Summarize the result by calendar quarter, customer zip code, and customer name. The result should include the grouped columns and full set of subtotals for every combination of grouped columns. Do not use the CUBE and UNION operators.

```
-- Query 2: Sales Order Shipments by Name, Zip, and Quarter
SELECT
   calQuarter,
   name,
   zip,
   SUM(EXTCOST) AS total_extended_cost,
   COUNT(transTypeKey) AS total_transactions
FROM INVENTORY_FACT inf, date_dim dd , CUST_VENDOR_DIM cd

WHERE
```

```
   dd.DATEKEY = inf.DATEKEY
   AND inf.CUSTVENDORKEY = cd.CUSTVENDORKEY
   AND dd.CalYear IN ('2011', '2012')
   AND inf.TRANSTYPEKEY = 5  -- Only shipments
GROUP BY ROLLUP
        (dd.calQuarter,cd.CUSTVENDORKEY, cd.ADDRCATCODE1,cd.name,cd.zip)
ORDER BY
        calQuarter , zip , Name ;
```

----------RESULTS:----------

History | Results

Result set 1 | Details

Filter table — Total:325

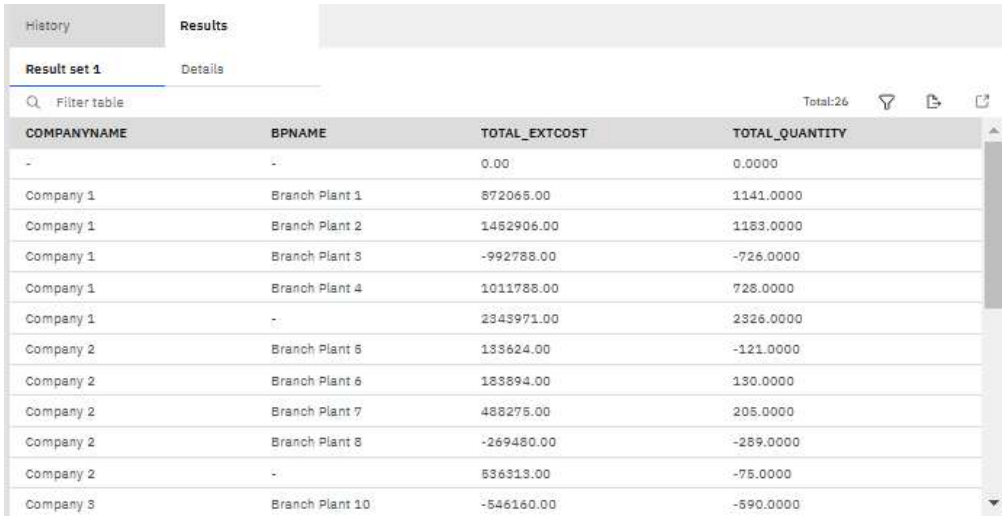
CALQUARTER	NAME	ZIP	TOTAL_EXTENDED_COST	TOTAL_TRANSACTIONS
1	Customer 11	02162	9082032.00	94
1	Customer 17	10231	10594965.00	103
1	Customer 2	10251	11075812.00	102
1	Customer 9	19084	12227671.00	114
1	Customer 12	20054	11000482.00	114
1	Customer 15	30385	10509024.00	101
1	Customer 13	33165	10095025.00	112
1	Customer 10	44136	11372861.00	110
1	Customer 1	60737	10298413.00	107
1	Customer 16	60926	8568012.00	88
1	Customer 7	75380	10447817.00	103

Query 3: Transfers by Company and Branch Plant

Write an SQL statement to display the sum of the extended cost and the sum of the quantity. The results should include data for transfers (transaction type 2). Summarize the result by company name and branch plant name. The result should include the grouped columns and a partial set of subtotals in order of the grouped columns (company name and branch plant name). Transfer quantities by design should sum to zero across all companies so that the grand total should be 0 for the sum of quantity and extended cost. Do not use the GROUPING SETS and UNION operators.

```
SELECT
        companyName,
        bpName,
   SUM(extCost) AS Total_ExtCost,
   SUM(quantity) AS Total_Quantity
FROM
   Company_Dim cd, Branch_Plant_Dim bd, Inventory_Fact inf
WHERE
        bd.CompanyKey = cd.CompanyKey
   AND inf.BranchPlantKey = bd.BranchPlantKey
   AND inf.TransTypeKey = 2
```

```
Group by ROLLUP (CompanyName,BPName)
ORDER BY
                cd.CompanyName NULLS FIRST,bd.BPName;
```



History | Results

Result set 1 | Details

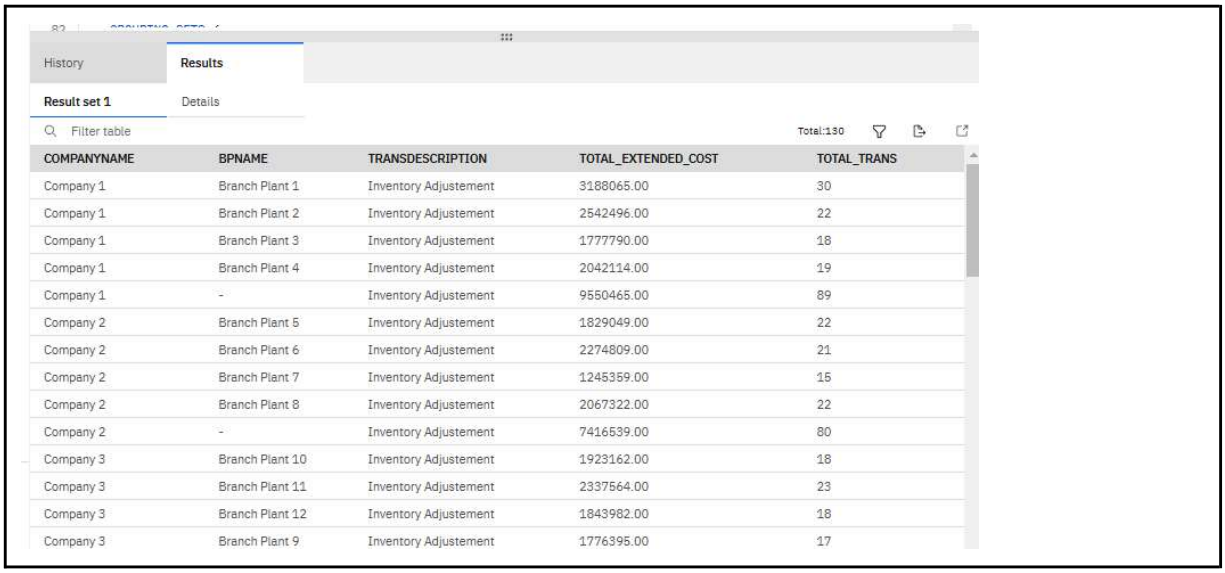
Filter table — Total:26

COMPANYNAME	BPNAME	TOTAL_EXTCOST	TOTAL_QUANTITY
-	-	0.00	0.0000
Company 1	Branch Plant 1	872065.00	1141.0000
Company 1	Branch Plant 2	1452906.00	1183.0000
Company 1	Branch Plant 3	-992788.00	-726.0000
Company 1	Branch Plant 4	1011788.00	728.0000
Company 1	-	2343971.00	2326.0000
Company 2	Branch Plant 5	133624.00	-121.0000
Company 2	Branch Plant 6	183894.00	130.0000
Company 2	Branch Plant 7	488275.00	205.0000
Company 2	Branch Plant 8	-269480.00	-289.0000
Company 2	-	536313.00	-75.0000
Company 3	Branch Plant 10	-546160.00	-590.0000

Query 4: Inventory Transactions by Transaction Description, Company, and Branch Plant

Write an SQL statement to display the sum of the extended cost and the number of inventory transactions. The results should include data for all transaction types. Summarize the result by transaction description, company name, and branch plant name. The result should include the grouped columns and partial totals in order of the grouped columns (transaction description, company name, and branch plant name). Do not use the ROLLUP and UNION operators.

```
SELECT
   CompanyName,
   BPName,
   TransDescription,
   SUM(extCost) AS Total_Extended_Cost,
   COUNT(*) AS Total_Trans
FROM
   company_dim cd, branch_plant_dim bd, inventory_Fact inf, trans_type_dim tr
WHERE
   cd.companyKey = bd.CompanyKey
   AND bd.branchPlantKey = inf.branchPlantKey
   AND inf.transtypekey = tr.transtypekey
GROUP BY
   GROUPING SETS (
      (cd.companyName, bd.BPName, tr.transDescription),
      (cd.companyName, tr.transDescription),
      (tr.transDescription)
   )
ORDER BY
   tr.transDescription, cd.companyName, bd.BPName;
```



History | Results

Result set 1 | Details

Filter table — Total:130

COMPANYNAME	BPNAME	TRANSDESCRIPTION	TOTAL_EXTENDED_COST	TOTAL_TRANS
Company 1	Branch Plant 1	Inventory Adjustement	3188065.00	30
Company 1	Branch Plant 2	Inventory Adjustement	2542496.00	22
Company 1	Branch Plant 3	Inventory Adjustement	1777790.00	18
Company 1	Branch Plant 4	Inventory Adjustement	2042114.00	19
Company 1	-	Inventory Adjustement	9550465.00	89
Company 2	Branch Plant 5	Inventory Adjustement	1829049.00	22
Company 2	Branch Plant 6	Inventory Adjustement	2274809.00	21
Company 2	Branch Plant 7	Inventory Adjustement	1245359.00	15
Company 2	Branch Plant 8	Inventory Adjustement	2067322.00	22
Company 2	-	Inventory Adjustement	7416539.00	80
Company 3	Branch Plant 10	Inventory Adjustement	1923162.00	18
Company 3	Branch Plant 11	Inventory Adjustement	2337564.00	23
Company 3	Branch Plant 12	Inventory Adjustement	1843982.00	18
Company 3	Branch Plant 9	Inventory Adjustement	1776395.00	17

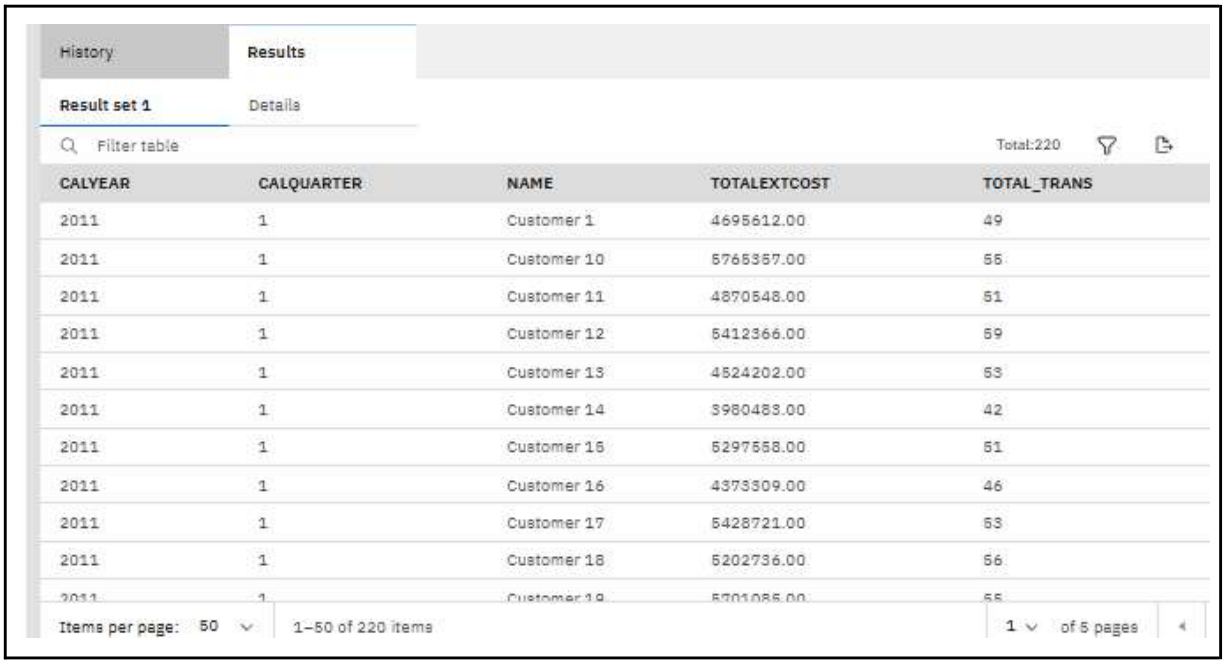
Query 5: Adjustments by Part Number

Write an SQL statement to display the sum of the extended cost and the number of inventory transactions. The results should include data for shipments (transaction type 5) in calendar years 2011 and 2012. Summarize the result by calendar year, calendar quarter, and customer name. The result should show the grouped columns and the normal set of group by results plus partial subtotals for year and quarter concatenated with customer name. Do not use the GROUPING SETS and UNION operators. (Hint: see the partial ROLLUP example in lesson 4).

```
--Query 5: Adjustments by Part Number



SELECT CalYear, CalQuarter, Name,
  sum(ExtCost) as TotalExtcost,
  COUNT(*) AS Total_Trans
FROM date_dim dd, inventory_fact inf, cust_vendor_dim cd
WHERE dd.DateKey = inf.DateKey
AND inf.CustVendorKey = cd.CustVendorKey
AND dd.CalYear in (2011,2012)
AND TransTypekey= 5
GROUP BY rollup (dd.CalYear, dd.CalQuarter), cd.Name
ORDER BY dd.CalYear, dd.CalQuarter, cd.Name;
```



History | Results

Result set 1 | Details

Filter table — Total:220

CALYEAR	CALQUARTER	NAME	TOTALEXTCOST	TOTAL_TRANS
2011	1	Customer 1	4695612.00	49
2011	1	Customer 10	5765367.00	55
2011	1	Customer 11	4870548.00	51
2011	1	Customer 12	5412366.00	59
2011	1	Customer 13	4524202.00	53
2011	1	Customer 14	3980483.00	42
2011	1	Customer 15	5297558.00	51
2011	1	Customer 16	4373309.00	46
2011	1	Customer 17	5428721.00	53
2011	1	Customer 18	5202736.00	56
2011	1	Customer 19	5701086.00	55

Items per page: 50 | 1–50 of 220 items | 1 of 5 pages

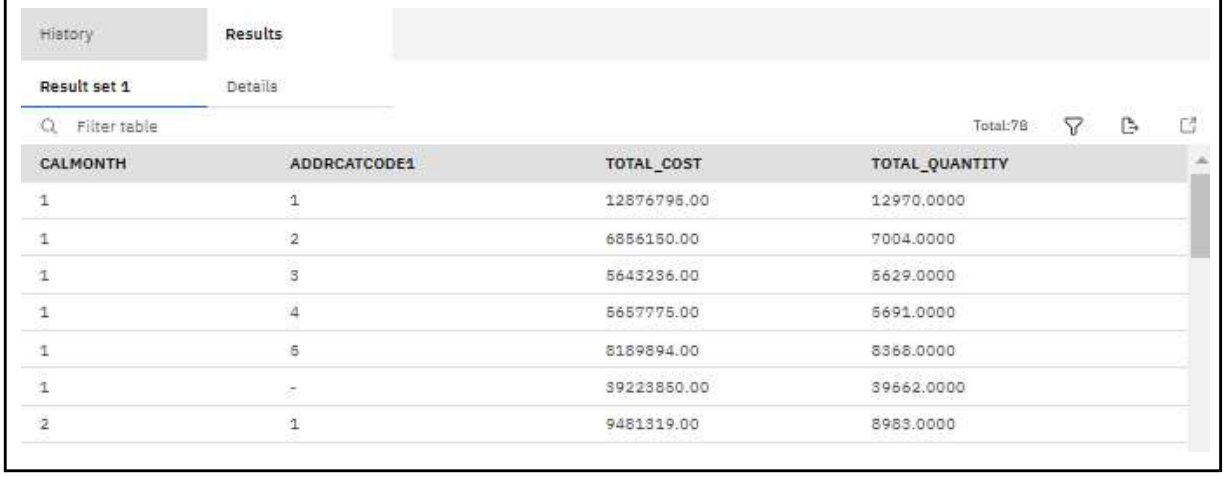
Query 6: Rewrite Query 1 without CUBE, ROLLUP, or GROUPING SETS

Rewrite query 1 without the usage of the CUBE ROLLUP, or GROUPING SETS operators. In rewriting the query, you should use NULL as the default value for each column.

```
--Query 6: Rewrite Query 1 without CUBE, ROLLUP, or GROUPING SETS

SELECT B.CalMonth, C.AddrCatCode1, sum(ExtCost) as total_cost, sum(Quantity) as total_quantity
FROM inventory_fact A, date_dim B, cust_vendor_dim C
WHERE A.DateKey = B.DateKey
AND A.CustVendorKey = C.CustVendorKey
AND TransTypeKey = 5
AND B.CalYear = 2011
GROUP BY B.CalMonth, C.AddrCatCode1
UNION
SELECT B.CalMonth, null, sum(ExtCost) as total_cost, sum(Quantity) as total_quantity
FROM inventory_fact A, date_dim B, cust_vendor_dim C
WHERE A.DateKey = B.DateKey
AND A.CustVendorKey = C.CustVendorKey
AND TransTypeKey = 5
AND B.CalYear = 2011
GROUP BY B.CalMonth
UNION
SELECT null, C.AddrCatCode1, sum(ExtCost) as total_cost, sum(Quantity) as total_quantity
FROM inventory_fact A, date_dim B, cust_vendor_dim C
WHERE A.DateKey = B.DateKey
AND A.CustVendorKey = C.CustVendorKey
```

```
AND TransTypeKey = 5
AND B.CalYear = 2011
GROUP BY C.AddrCatCode1
UNION
SELECT null, null, sum(ExtCost) as total_cost, sum(Quantity) as total_quantity
FROM inventory_fact A, date_dim B, cust_vendor_dim C
WHERE A.DateKey = B.DateKey
AND A.CustVendorKey = C.CustVendorKey
AND TransTypeKey = 5
AND B.CalYear = 2011
ORDER BY 1,2;
```


History Results

Result set 1 Details

Filter table Total:78

CALMONTH	ADDRCATCODE1	TOTAL_COST	TOTAL_QUANTITY
1	1	12876795.00	12970.0000
1	2	6856150.00	7004.0000
1	3	5643236.00	5629.0000
1	4	5657775.00	5691.0000
1	5	8189894.00	8368.0000
1	-	39223850.00	39662.0000
2	1	9481319.00	8983.0000

Query 7: Rewrite Query 3 without CUBE, ROLLUP, or GROUPING SETS

Rewrite query 3 without the usage of the CUBE, ROLLUP, or GROUPING SETS operators. In rewriting the query, you should use NULL as the default value for each column.

```
--Query 7------

SELECT
            companyName,
            bd.bpName,
   SUM(extCost) AS Total_ExtCost,
   SUM(quantity) AS Total_Quantity
FROM
   company_Dim cd, Branch_Plant_Dim bd, Inventory_Fact inf
WHERE
            bd.CompanyKey = cd.CompanyKey
   AND inf.BranchPlantKey = bd.BranchPlantKey
   AND inf.TransTypeKey = 2
Group by cd.CompanyName, bd.BPName
UNION
SELECT
            cd.CompanyName,
```

```
            NULL,
    SUM(inf.extCost) AS Total_ExtCost,
    SUM(inf.Quantity) AS Total_Quantity
FROM
    Company_Dim cd, Branch_Plant_Dim bd, Inventory_Fact inf
WHERE
            bd.CompanyKey = cd.CompanyKey
    AND inf.BranchPlantKey = bd.BranchPlantKey
    AND inf.TransTypeKey = 2
Group by cd.CompanyName
UNION
SELECT
            NULL,
            bd.BPName,
    SUM(inf.extCost) AS Total_ExtCost,
    SUM(inf.Quantity) AS Total_Quantity
FROM
    Company_Dim cd, Branch_Plant_Dim bd, Inventory_Fact inf
WHERE
            bd.CompanyKey = cd.CompanyKey
    AND inf.BranchPlantKey = bd.BranchPlantKey
    AND inf.TransTypeKey = 2
Group by bd.BPName
ORDER BY
                    1 ,2;
```

History | Results

Result set 1 | Details

Filter table — Total:45

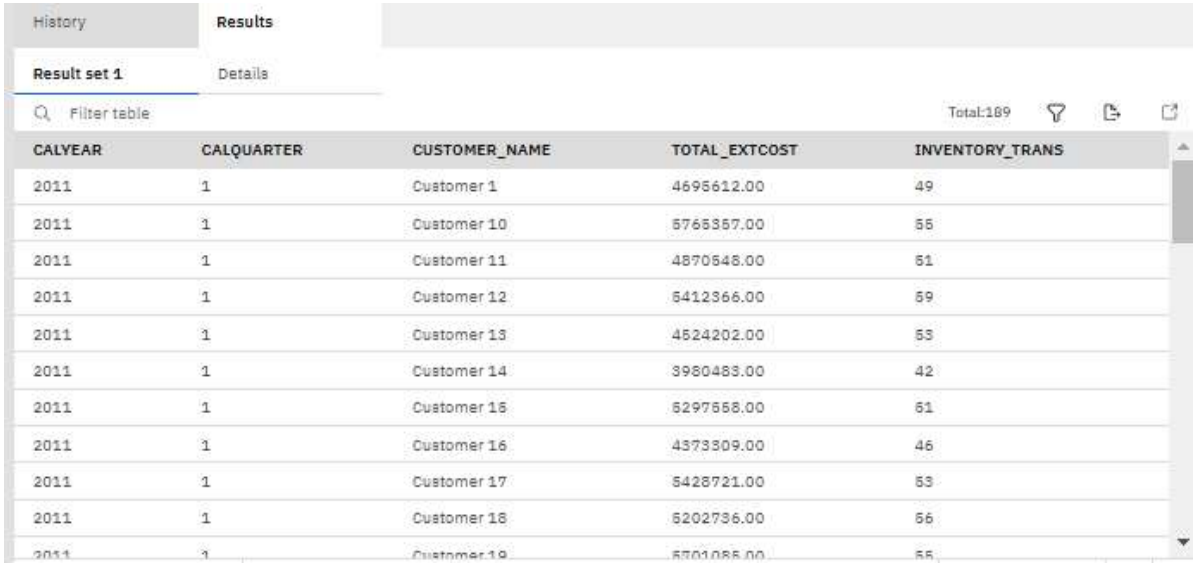
COMPANYNAME	BPNAME	TOTAL_EXTCOST	TOTAL_QUANTITY
Company 1	Branch Plant 1	872065.00	1141.0000
Company 1	Branch Plant 2	1452906.00	1183.0000
Company 1	Branch Plant 3	-992788.00	-726.0000
Company 1	Branch Plant 4	1011788.00	728.0000
Company 1	-	2343971.00	2326.0000
Company 2	Branch Plant 5	133624.00	-121.0000
Company 2	Branch Plant 6	183894.00	130.0000
Company 2	Branch Plant 7	488275.00	205.0000
Company 2	Branch Plant 8	-269480.00	-289.0000
Company 2	-	536313.00	-75.0000

Query 8: Sales Order Shipments by Name and Combination of Year and Quarter

Write an SQL statement to display the sum of the extended cost and the number of inventory transactions. The results should include data for shipments (transaction type 5) in calendar years 2011 and 2012. Summarize the result by calendar year, calendar quarter, and customer name. The result should include

the grouped columns and the full set of subtotals for customer name and the combination of year and quarter. Do not use the GROUPING SETS and UNION operators. (Hint: see the composite column example in lesson 4).

```
--Query 8: Sales Order Shipments by Name and Combination of Year and Quarter
SELECT
        calYear,
        calQuarter ,
        Name AS Customer_Name,
        SUM(extCost) as Total_ExtCost,
        COUNT (inventoryKey) AS Inventory_Trans
FROM
        inventory_fact inf,date_dim dd,cust_vendor_dim cd
WHERE
        --group column and full set column
        inf.dateKey = dd.dateKey
        AND inf.CustVendorKey = cd.CustVendorKey
        AND inf.TransTypeKey = 5
        AND dd.calyear in (2011,2012)
GROUP BY CUBE (cd.name,(dd.calYear,dd.calQuarter))
ORDER BY dd.calYear, dd.calQuarter, cd.name;
```



History | Results

Result set 1 | Details

Filter table — Total:189

CALYEAR	CALQUARTER	CUSTOMER_NAME	TOTAL_EXTCOST	INVENTORY_TRANS
2011	1	Customer 1	4695612.00	49
2011	1	Customer 10	5765357.00	55
2011	1	Customer 11	4870548.00	51
2011	1	Customer 12	6412366.00	59
2011	1	Customer 13	4524202.00	53
2011	1	Customer 14	3980483.00	42
2011	1	Customer 15	6297668.00	61
2011	1	Customer 16	4373309.00	46
2011	1	Customer 17	5428721.00	53
2011	1	Customer 18	6202736.00	66
2011	1	Customer 19	5701086.00	55

Query 9: Sales Order Shipments by Month and Category Code1 with Group Number

Write an SQL statement to display the sum of the extended cost and the sum of the quantity. The results should include data for shipments (transaction type 5) in calendar year 2011. Summarize the result by calendar month and Address Category Code 1. The result should include the grouped columns and the full

set of subtotals for every combination of grouped columns along with the hierarchical group number for both grouping columns. Do not use the GROUPING SETS and UNION operators. (Hint: see the group functions slide in lesson 5).

```
--Query 9: Sales Order Shipments by Month and Category Code1 with Group Number
SELECT
   calMonth,
   addrCatCode1,
   SUM(extCost) AS Total_ExtCost,
   SUM(quantity) AS Quantity,
   (GROUPING(calMonth) * 2) + GROUPING(addrCatCode1) AS Group_Level
FROM
   inventory_fact A, date_dim B, cust_vendor_dim C
WHERE
   A.DateKey = B.DateKey
   AND A.CustVendorKey = C.CustVendorKey
   AND TranstypeKey = 5
   AND B.CalYear = 2011
GROUP BY
   CUBE(calMonth, addrCatCode1)
ORDER BY
   Group_Level;
```

ory Results

ult set 1 Details

Filter table Total:78

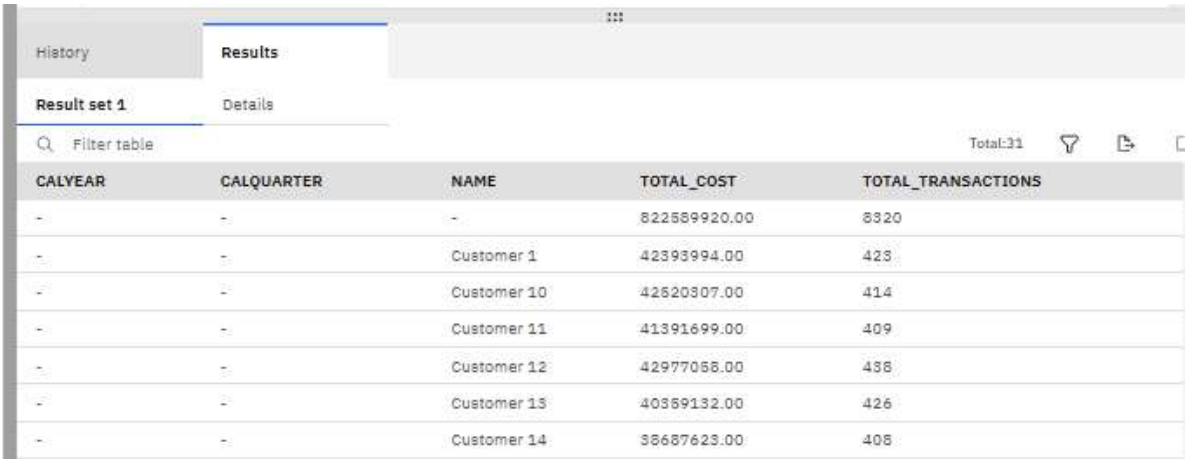
MONTH	ADDRCATCODE1	TOTAL_EXTCOST	QUANTITY	GROUP_LEVEL
	1	12876795.00	12970.0000	0
	2	6856150.00	7004.0000	0
	3	5643236.00	5629.0000	0
	4	5657775.00	5691.0000	0
	5	8189894.00	8368.0000	0
	1	9481319.00	8983.0000	0
	2	6808224.00	6935.0000	0
	3	4023869.00	4139.0000	0
	4	4862270.00	4657.0000	0

ns per page: 50 1–50 of 78 items 1 of 2 pages

Query 10: Sales Order Shipments with Subtotals by Name and Partial Subtotals by Year and Quarter

Write an SQL statement to display the sum of the extended cost and the number of inventory transactions. The results should include data for shipments (transaction type 5) in calendar years 2011 and 2012. Summarize the result by calendar year, calendar quarter, and customer name. The result should include the grouped columns and subtotals for customer name along with partial subtotals for year and quarter. Do not include the normal GROUP BY totals in the result. Do not use the UNION operator. (Hint: see the nested rollup example in lesson 4).

```
SELECT
    calYear,
    calQuarter,
    name,
    SUM(extCost) AS total_cost,
    COUNT(inventoryKey) AS total_transactions
FROM
    inventory_fact A,
    date_dim B,
    cust_vendor_dim C
WHERE
    A.dateKey = B.dateKey
    AND A.custVendorKey = C.custVendorKey
    AND transTypeKey = 5
    AND B.calYear IN (2011, 2012)
GROUP BY
    GROUPING SETS (ROLLUP(calYear, calQuarter), name)
ORDER BY
    calYear, calQuarter, name;
```



History | Results

Result set 1 | Details

Filter table — Total:31

CALYEAR	CALQUARTER	NAME	TOTAL_COST	TOTAL_TRANSACTIONS
-	-	-	822589920.00	8320
-	-	Customer 1	42393994.00	423
-	-	Customer 10	42620307.00	414
-	-	Customer 11	41391699.00	409
-	-	Customer 12	42977068.00	438
-	-	Customer 13	40359132.00	426
-	-	Customer 14	38687623.00	408


18,300
Rows read
18,300
Rows loaded
0
Rows rejected
Start time
02/15/2025 10:50:03 AM
End time
02/15/2025 10:50:09 AM
The data load job succeeded.
You can now work with your data.